

Nombre Alumno: Enzo Romero DNI:41948148.

Nombre Alumno: Francisco Morales DNI: 43533393.

Nombre Profesor: **Yanina Medina – Andrea Lezcano Airdi - Matías Mascazzini**

Grupo Laboratorio: Comisión 2.

TP: Trabajo Practico N°4.

Fecha de Entrega: Domingo 20/09/26.

Informe d Fundamentación Teórica y Meta-Análisis.

Tema: Colecciones de objetos estáticas y Dinamicas

1) Introducción y objetivos:

El presente trabajo práctico aborda la transición del manejo de objetos individuales al trabajo con agrupaciones o conjuntos de elementos en Java. En el desarrollo de software real, es indispensable gestionar listas de datos que puedan crecer, achicarse o filtrarse según las necesidades de la aplicación.

A lo largo de este práctico se busca aprender a seleccionar la estructura de datos adecuada para cada escenario. Se analizan desde contenedores de tamaño fijo (arreglos) hasta las principales colecciones dinámicas del *Java Collections Framework* (ArrayList, HashSet y HashMap). Asimismo, el práctico exige aplicar la reutilización de clases modeladas previamente (Punto, Producto, Cliente, Empleado, Persona, Localidad), reforzando el concepto de modularidad en la programación orientada a objetos

2). Fundamentación de Conceptos Teóricos y Ejemplos de Aplicación

Para el primer ejercicio (Array de Puntos) se implemento una colección (o contenedor) de tipo estatico y homogénea

Mas precisamente un contenedor de tipo Punto con espacio para objetos de tipo punto es decir contenedor homogeneo de tipo estatico.

Colecciones donde importa el orden de llegada y se permiten elementos repetidos (como la nómina de empleados en Banco o la lista de productos en una Factura).

```
//declaro e instancio el contenedor estatico de 6 posiciones
Punto [] puntos= new Punto [6];
// ----- de datos con prueba
```

Implementación de una Lista Homogénea Dinámica (ArrayList)

En Java, un **ArrayList** representa una colección **dinámica y homogénea**:

- **Dinámica:** A diferencia de un arreglo convencional (como Punto[]), su tamaño no es fijo. Puede crecer o reducirse en tiempo de ejecución a medida que se agregan o quitan elementos mediante métodos como .add() y .remove().
- **Homogénea:** Aunque internamente gestiona los elementos como Object, está concebida para almacenar objetos de un mismo tipo conceptual (en este caso, objetos de la clase Producto).

```
private ArrayList productos; //
```

Métodos para manipular el array list:

```

79     public boolean agregarProducto(Producto p_producto){
80         return this.getProductos().add(p_producto);
81     }
82 }
83 /**
84  * @param p_producto Objeto Prudcto a remover
85  * @return true si el producto existia y se removio
86  */
87 public boolean quitarProducto(Producto p_producto){
88     return this.getProductos().remove(p_producto);
89 }

```

Implementación de un hash set conjunto no ordenado de objetos que permite búsquedas veloces calculando índices internos mediante códigos hash.

```

* Devuelve un HashSet con todos los titulares únicos de las cuentas bancarias.
*/
public HashSet<Persona> listaDeTitulares() {
    HashSet<Persona> titulares = new HashSet<Persona>();
    for (Object obj : this.getCuentasBancarias()) {
        CuentaBancaria cb = (CuentaBancaria) obj;
        titulares.add(cb.getTitular()); // HashSet descarta automáticamente los
    }
    return titulares;
}

```

Implementación de un hash map estructura para almacenar pares clave valor indexando el valor en base al hash code de su clave.

```

public class Curso {
    private String nombre;
    private HashMap alumnos; // <Integer (LU), Alumno>

    public Curso(String p_nombre) {
        this.setNombre(p_nombre);
        this.setAlumnos(new HashMap());
    }
}

```

Conclusión:

Este trabajo practico numero 4 profundo en temas nuevos (colecciones o contenedores) como se diferencian entre estáticos o dinámicos, homogéneos o heterogéneos y también su aplicación mostrándonos junto con la interacción entre clases y objetos como funciona.